

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.ensemble import GradientBoostingRegressor
print("Libraries imported sucessfully")
```

Libraries imported sucessfully

```
# Load datasets
```

```
sales = pd.read_csv("/kaggle/bm_sales.csv")
skus = pd.read_csv("/kaggle/bm_skus.csv")
promotions = pd.read_csv("/kaggle/bm_promotions.csv")
inventory = pd.read_csv("/kaggle/bm_inventory.csv")

# Convert date columns
sales["date"] = pd.to_datetime(sales["date"])
promotions["start_date"] = pd.to_datetime(promotions["start_date"])
promotions["end_date"] = pd.to_datetime(promotions["end_date"])
inventory["last_restock_date"] = pd.to_datetime(inventory["last_restock_date"])
inventory["snapshot_date"] = pd.to_datetime(inventory["snapshot_date"])

print("Datasets loaded successfully!")

print("\nSales shape:", sales.shape)
print("SKU shape:", skus.shape)
print("Promotions shape:", promotions.shape)
print("Inventory shape:", inventory.shape)
```

Datasets loaded successfully!

```
Sales shape: (641843, 9)
SKU shape: (200, 7)
Promotions shape: (33, 6)
Inventory shape: (8735, 7)
```

```
# Merge sales with SKU master
```

```
sales_sku = sales.merge(
    skus,
    on="sku_id",
    how="left"
)

# Create weekly time period
sales_sku["week"] = (
    sales_sku["date"]
    .dt.to_period("W-SUN")
    .dt.start_time
)
```

```
# Aggregate daily sales into weekly SKU-level demand
weekly_sales = (
    sales_sku
    .groupby(["week", "sku_id"], as_index=False)
    .agg(
        demand=("quantity", "sum"),
        sales_value=("total_value", "sum"),
        avg_price=("unit_price_x", "mean"),
        avg_discount=("discount_pct", "mean")
    )
)

print("Weekly SKU-level dataset created!")
print("Shape:", weekly_sales.shape)
print("Number of weeks:", weekly_sales["week"].nunique())
print("Number of SKUs:", weekly_sales["sku_id"].nunique())

print("\nDate range:")
print(weekly_sales["week"].min(), "to", weekly_sales["week"].max())

print("\nFirst 5 rows:")
print(weekly_sales.head())
```

Weekly SKU-level dataset created!

Shape: (50600, 6)

Number of weeks: 253

Number of SKUs: 200

Date range:

2020-12-28 00:00:00 to 2025-10-27 00:00:00

First 5 rows:

	week	sku_id	demand	sales_value	avg_price	avg_discount
0	2020-12-28	1001	39	5751.48	146.931429	10.714286
1	2020-12-28	1002	35	435.23	12.585556	8.333333
2	2020-12-28	1003	42	469.80	11.169231	6.923077
3	2020-12-28	1004	15	342.54	22.960000	7.500000
4	2020-12-28	1005	17	282.28	16.560000	6.428571

```
# Create a complete SKU × week grid
```

```
all_weeks = pd.date_range(
    start=weekly_sales["week"].min(),
    end=weekly_sales["week"].max(),
    freq="7D"
)
```

```
all_skus = weekly_sales["sku_id"].unique()
```

```
complete_grid = pd.MultiIndex.from_product(
    [all_weeks, all_skus],
    names=["week", "sku_id"]
).to_frame(index=False)
```

```
# Merge weekly sales onto the complete grid
```

```

weekly_sales_complete = complete_grid.merge(
    weekly_sales,
    on=["week", "sku_id"],
    how="left"
)

print("Complete SKU-week dataset created!")
print("Shape:", weekly_sales_complete.shape)
print("Number of weeks:", weekly_sales_complete["week"].nunique())
print("Number of SKUs:", weekly_sales_complete["sku_id"].nunique())

print("\nMissing values:")
print(weekly_sales_complete.isnull().sum())

```

```

Complete SKU-week dataset created!
Shape: (50600, 6)
Number of weeks: 253
Number of SKUs: 200

```

```

Missing values:
week          0
sku_id        0
demand        0
sales_value   0
avg_price     0
avg_discount  0
dtype: int64

```

```

# Add SKU information

weekly_sales_complete = weekly_sales_complete.merge(
    skus[
        [
            "sku_id",
            "sku_name",
            "category",
            "subcategory",
            "brand",
            "cost_price"
        ]
    ],
    on="sku_id",
    how="left"
)

print("SKU information added successfully!")

print("Shape:", weekly_sales_complete.shape)

print("\nMissing values:")
print(
    weekly_sales_complete[
        [
            "sku_name",
            "category",

```

```

        "subcategory",
        "brand",
        "cost_price"
    ]
    ].isnull().sum()
)

print("\nFirst 5 rows:")
print(weekly_sales_complete.head())

```

SKU information added successfully!
Shape: (50600, 11)

Missing values:
sku_name 0
category 0
subcategory 0
brand 0
cost_price 0
dtype: int64

First 5 rows:

	week	sku_id	demand	sales_value	avg_price	avg_discount	\
0	2020-12-28	1001	39	5751.48	146.931429	10.714286	
1	2020-12-28	1002	35	435.23	12.585556	8.333333	
2	2020-12-28	1003	42	469.80	11.169231	6.923077	
3	2020-12-28	1004	15	342.54	22.960000	7.500000	
4	2020-12-28	1005	17	282.28	16.560000	6.428571	

	sku_name	category	subcategory	\
0	Electronics_Chargers_1001	Electronics	Chargers	
1	Dairy_Cream_1002	Dairy	Cream	
2	Household_Cleaning_Supplies_1003	Household	Cleaning Supplies	
3	Beverages_Juices_1004	Beverages	Juices	
4	Household_Paper_Products_1005	Household	Paper Products	

	brand	cost_price
0	Budget	103.44
1	Local	8.96
2	BlueMart	7.47
3	International	17.58
4	Budget	13.68

Create calendar features

```

weekly_sales_complete["week_number"] = (
    weekly_sales_complete["week"]
    .dt.isocalendar()
    .week
    .astype(int)
)

weekly_sales_complete["month"] = weekly_sales_complete["week"].dt.month
weekly_sales_complete["quarter"] = weekly_sales_complete["week"].dt.quarter
weekly_sales_complete["year"] = weekly_sales_complete["week"].dt.year

print("Calendar features created successfully!")

```

```

print("\nCalendar features:")
print(
    weekly_sales_complete[
        ["week", "week_number", "month", "quarter", "year"]
    ].head(10)
)

print("\nMissing values:")
print(
    weekly_sales_complete[
        ["week_number", "month", "quarter", "year"]
    ].isnull().sum()
)

```

Calendar features created successfully!

Calendar features:

	week	week_number	month	quarter	year
0	2020-12-28	53	12	4	2020
1	2020-12-28	53	12	4	2020
2	2020-12-28	53	12	4	2020
3	2020-12-28	53	12	4	2020
4	2020-12-28	53	12	4	2020
5	2020-12-28	53	12	4	2020
6	2020-12-28	53	12	4	2020
7	2020-12-28	53	12	4	2020
8	2020-12-28	53	12	4	2020
9	2020-12-28	53	12	4	2020

Missing values:

```

week_number    0
month          0
quarter        0
year           0
dtype: int64

```

Create lag features

```
lag_periods = [1, 2, 4, 8, 13, 26, 52]
```

```

for lag in lag_periods:
    weekly_sales_complete[f"lag_{lag}"] = (
        weekly_sales_complete
        .groupby("sku_id")["demand"]
        .shift(lag)
    )

```

print("Lag features created successfully!")

print("\nLag features:")

```

print(
    weekly_sales_complete[
        ["sku_id", "week", "demand",
         "lag_1", "lag_2", "lag_4",
         "lag_8", "lag_13", "lag_26", "lag_52"]
    ]
)

```

```

    ].head(15)
)

print("\nMissing values in lag features:")
print(
    weekly_sales_complete[
        [f"lag_{lag}" for lag in lag_periods]
    ].isnull().sum()
)

```

Lag features created successfully!

Lag features:

	sku_id	week	demand	lag_1	lag_2	lag_4	lag_8	lag_13	lag_26	\
0	1001	2020-12-28	39	NaN	NaN	NaN	NaN	NaN	NaN	
1	1002	2020-12-28	35	NaN	NaN	NaN	NaN	NaN	NaN	
2	1003	2020-12-28	42	NaN	NaN	NaN	NaN	NaN	NaN	
3	1004	2020-12-28	15	NaN	NaN	NaN	NaN	NaN	NaN	
4	1005	2020-12-28	17	NaN	NaN	NaN	NaN	NaN	NaN	
5	1006	2020-12-28	27	NaN	NaN	NaN	NaN	NaN	NaN	
6	1007	2020-12-28	41	NaN	NaN	NaN	NaN	NaN	NaN	
7	1008	2020-12-28	24	NaN	NaN	NaN	NaN	NaN	NaN	
8	1009	2020-12-28	26	NaN	NaN	NaN	NaN	NaN	NaN	
9	1010	2020-12-28	9	NaN	NaN	NaN	NaN	NaN	NaN	
10	1011	2020-12-28	38	NaN	NaN	NaN	NaN	NaN	NaN	
11	1012	2020-12-28	18	NaN	NaN	NaN	NaN	NaN	NaN	
12	1013	2020-12-28	39	NaN	NaN	NaN	NaN	NaN	NaN	
13	1014	2020-12-28	31	NaN	NaN	NaN	NaN	NaN	NaN	
14	1015	2020-12-28	26	NaN	NaN	NaN	NaN	NaN	NaN	

lag_52

0	NaN
1	NaN
2	NaN
3	NaN
4	NaN
5	NaN
6	NaN
7	NaN
8	NaN
9	NaN
10	NaN
11	NaN
12	NaN
13	NaN
14	NaN

Missing values in lag features:

lag_1	200
lag_2	400
lag_4	800
lag_8	1600
lag_13	2600
lag_26	5200
lag_52	10400

dtype: int64

```

# Create leakage-safe rolling mean features

rolling_windows = [4, 8, 13]

for window in rolling_windows:
    weekly_sales_complete[f"rolling_mean_{window}"] = (
        weekly_sales_complete
        .groupby("sku_id")["demand"]
        .transform(
            lambda x: x.shift(1).rolling(window).mean()
        )
    )

print("Rolling mean features created successfully!")

print("\nRolling mean features:")
print(
    weekly_sales_complete[
        [
            "sku_id",
            "week",
            "demand",
            "rolling_mean_4",
            "rolling_mean_8",
            "rolling_mean_13"
        ]
    ].head(20)
)

print("\nMissing values in rolling features:")
print(
    weekly_sales_complete[
        [
            "rolling_mean_4",
            "rolling_mean_8",
            "rolling_mean_13"
        ]
    ].isnull().sum()
)

```

Rolling mean features created successfully!

Rolling mean features:

	sku_id	week	demand	rolling_mean_4	rolling_mean_8	rolling_mean_13
0	1001	2020-12-28	39	NaN	NaN	Na
1	1002	2020-12-28	35	NaN	NaN	Na
2	1003	2020-12-28	42	NaN	NaN	Na
3	1004	2020-12-28	15	NaN	NaN	Na
4	1005	2020-12-28	17	NaN	NaN	Na
5	1006	2020-12-28	27	NaN	NaN	Na
6	1007	2020-12-28	41	NaN	NaN	Na
7	1008	2020-12-28	24	NaN	NaN	Na
8	1009	2020-12-28	26	NaN	NaN	Na
9	1010	2020-12-28	9	NaN	NaN	Na
10	1011	2020-12-28	38	NaN	NaN	Na
11	1012	2020-12-28	18	NaN	NaN	Na

12	1013	2020-12-28	39	NaN	NaN	Na
13	1014	2020-12-28	31	NaN	NaN	Na
14	1015	2020-12-28	26	NaN	NaN	Na
15	1016	2020-12-28	21	NaN	NaN	Na
16	1017	2020-12-28	32	NaN	NaN	Na
17	1018	2020-12-28	14	NaN	NaN	Na
18	1019	2020-12-28	21	NaN	NaN	Na
19	1020	2020-12-28	29	NaN	NaN	Na

Missing values in rolling features:

```
rolling_mean_4      800
rolling_mean_8     1600
rolling_mean_13    2600
dtype: int64
```

```
# Create weekly promotion features
```

```
promo_weeks = []
```

```
for _, promo in promotions.iterrows():
```

```
    weeks = pd.date_range(
        start=promo["start_date"].to_period("W-SUN").start_time,
        end=promo["end_date"].to_period("W-SUN").start_time,
        freq="7D"
    )
```

```
    for week in weeks:
        promo_weeks.append({
            "week": week,
            "promo_type": promo["promo_type"],
            "discount_pct": promo["discount_pct"]
        })
```

```
promo_weekly = pd.DataFrame(promo_weeks)
```

```
print("Weekly promotion data created!")
```

```
print("Shape:", promo_weekly.shape)
```

```
print("\nFirst 10 rows:")
```

```
print(promo_weekly.head(10))
```

Weekly promotion data created!

Shape: (141, 3)

First 10 rows:

	week	promo_type	discount_pct
0	2021-04-12	Ramadan	25
1	2021-04-19	Ramadan	25
2	2021-04-26	Ramadan	25
3	2021-05-03	Ramadan	25
4	2021-05-10	Ramadan	25
5	2021-05-10	Eid	20
6	2021-07-19	Eid	25
7	2020-12-28	DSF	15

8	2021-01-04	DSF	15
9	2021-01-11	DSF	15

Double-click (or enter) to edit

```
# Aggregate promotions to one row per week

promo_weekly_agg = (
    promo_weekly
    .groupby("week", as_index=False)
    .agg(
        promo_flag=("promo_type", lambda x: 1),
        promo_discount=("discount_pct", "max"),
        promo_count=("promo_type", "count")
    )
)

print("Promotion features aggregated successfully!")

print("Shape:", promo_weekly_agg.shape)

print("\nNumber of unique promotion weeks:",
      promo_weekly_agg["week"].nunique())

print("\nDuplicate weeks:",
      promo_weekly_agg["week"].duplicated().sum())

print("\nFirst 10 rows:")
print(promo_weekly_agg.head(10))
```

Promotion features aggregated successfully!
Shape: (132, 4)

Number of unique promotion weeks: 132

Duplicate weeks: 0

First 10 rows:

	week	promo_flag	promo_discount	promo_count
0	2020-12-28	1	15	1
1	2021-01-04	1	15	1
2	2021-01-11	1	15	1
3	2021-01-18	1	15	1
4	2021-01-25	1	15	1
5	2021-02-01	1	15	1
6	2021-02-08	1	15	1
7	2021-02-15	1	15	1
8	2021-02-22	1	15	1
9	2021-04-12	1	25	1

```
# Merge weekly promotion features
```

```
weekly_sales_complete = weekly_sales_complete.merge(
    promo_weekly_agg,
    on="week",
```

```

        how="left"
    )

    print("Promotion features merged successfully!")

    print("Shape:", weekly_sales_complete.shape)

    print("\nMissing promotion values:")
    print(
        weekly_sales_complete[
            ["promo_flag", "promo_discount", "promo_count"]
        ].isnull().sum()
    )

```

```

Promotion features merged successfully!
Shape: (50600, 28)

```

```

Missing promotion values:
promo_flag      24200
promo_discount   24200
promo_count      24200
dtype: int64

```

```

# Fill missing promotion values for non-promotion weeks

weekly_sales_complete["promo_flag"] = (
    weekly_sales_complete["promo_flag"].fillna(0)
)

weekly_sales_complete["promo_discount"] = (
    weekly_sales_complete["promo_discount"].fillna(0)
)

weekly_sales_complete["promo_count"] = (
    weekly_sales_complete["promo_count"].fillna(0)
)

print("Missing promotion values filled successfully!")

print("\nMissing values:")
print(
    weekly_sales_complete[
        ["promo_flag", "promo_discount", "promo_count"]
    ].isnull().sum()
)

print("\nPromotion flag distribution:")
print(weekly_sales_complete["promo_flag"].value_counts())

```

```

Missing promotion values filled successfully!

```

```

Missing values:
promo_flag      0
promo_discount   0

```

```
promo_count      0
dtype: int64
```

Promotion flag distribution:

```
promo_flag
1.0      26400
0.0      24200
```

```
Name: count, dtype: int64
```

```
# Define forecasting features
```

```
feature_columns = [
    "lag_1",
    "lag_2",
    "lag_4",
    "lag_8",
    "lag_13",
    "lag_26",
    "lag_52",
    "rolling_mean_4",
    "rolling_mean_8",
    "rolling_mean_13",
    "week_number",
    "month",
    "quarter",
    "year",
    "promo_flag",
    "promo_discount",
    "promo_count"
]
```

```
print("Number of features:", len(feature_columns))
```

```
print("\nFeatures used by the model:")
for feature in feature_columns:
    print("-", feature)
```

Number of features: 17

Features used by the model:

- lag_1
- lag_2
- lag_4
- lag_8
- lag_13
- lag_26
- lag_52
- rolling_mean_4
- rolling_mean_8
- rolling_mean_13
- week_number
- month
- quarter
- year
- promo_flag
- promo_discount
- promo_count

```

# Create model-ready dataset
# Remove rows where required lag/rolling features are unavailable

model_ready = weekly_sales_complete.dropna(
    subset=feature_columns
).copy()

print("Model-ready dataset created!")

print("Shape:", model_ready.shape)

print("\nDate range:")
print("Start:", model_ready["week"].min())
print("End:", model_ready["week"].max())

print("\nNumber of weeks:",
      model_ready["week"].nunique())

print("Number of SKUs:",
      model_ready["sku_id"].nunique())

print("\nMissing values in model features:")
print(
    model_ready[feature_columns].isnull().sum().sum()
)

```

```

Model-ready dataset created!
Shape: (40200, 28)

Date range:
Start: 2021-12-27 00:00:00
End: 2025-10-27 00:00:00

Number of weeks: 201
Number of SKUs: 200

Missing values in model features:
0

```

```

# Define the final 6 weeks as the test period

test_weeks = sorted(
    model_ready["week"].unique()
)[-6:]

print("Test weeks:")
for week in test_weeks:
    print(week)

print("\nNumber of test weeks:", len(test_weeks))

```

```

Test weeks:
2025-09-22 00:00:00
2025-09-29 00:00:00

```

2025-10-06 00:00:00
2025-10-13 00:00:00
2025-10-20 00:00:00
2025-10-27 00:00:00

Number of test weeks: 6

```
# Time-based train/test split

train_data = model_ready[
    ~model_ready["week"].isin(test_weeks)
].copy()

test_data = model_ready[
    model_ready["week"].isin(test_weeks)
].copy()

# Create X and y

X_train = train_data[feature_columns]
y_train = train_data["demand"]

X_test = test_data[feature_columns]
y_test = test_data["demand"]

print("Train/Test split completed!")

print("\nTraining data:")
print("X_train:", X_train.shape)
print("y_train:", y_train.shape)

print("\nTest data:")
print("X_test:", X_test.shape)
print("y_test:", y_test.shape)

print("\nTraining period:")
print(train_data["week"].min(), "to", train_data["week"].max())

print("\nTest period:")
print(test_data["week"].min(), "to", test_data["week"].max())
```

Train/Test split completed!

Training data:
X_train: (39000, 17)
y_train: (39000,)

Test data:
X_test: (1200, 17)
y_test: (1200,)

Training period:
2021-12-27 00:00:00 to 2025-09-15 00:00:00

Test period:
2025-09-22 00:00:00 to 2025-10-27 00:00:00

```

# Seasonal Naive Baseline
# Forecast = demand from 52 weeks ago

baseline = weekly_sales_complete[
    ["week", "sku_id", "demand"]
].copy()

baseline = baseline.sort_values(
    ["sku_id", "week"]
)

baseline["forecast"] = (
    baseline
    .groupby("sku_id")["demand"]
    .shift(52)
)

# Keep only the 6-week test period
baseline_test = baseline[
    baseline["week"].isin(test_weeks)
].copy()

print("Seasonal Naive baseline created!")

print("\nBaseline test shape:",
      baseline_test.shape)

print("\nMissing baseline forecasts:",
      baseline_test["forecast"].isnull().sum())

print("\nFirst 10 baseline predictions:")
print(baseline_test.head(10))

```

Seasonal Naive baseline created!

Baseline test shape: (1200, 4)

Missing baseline forecasts: 0

First 10 baseline predictions:

	week	sku_id	demand	forecast
49400	2025-09-22	1001	12	21.0
49600	2025-09-29	1001	15	20.0
49800	2025-10-06	1001	25	8.0
50000	2025-10-13	1001	13	23.0
50200	2025-10-20	1001	25	21.0
50400	2025-10-27	1001	20	22.0
49401	2025-09-22	1002	17	20.0
49601	2025-09-29	1002	27	23.0
49801	2025-10-06	1002	22	30.0
50001	2025-10-13	1002	10	24.0

```
# Evaluate Seasonal Naive Baseline
```

```
actual = baseline_test["demand"]
```

```

forecast = baseline_test["forecast"]

wape_baseline = (
    np.sum(np.abs(actual - forecast))
    / np.sum(actual)
) * 100

bias_baseline = (
    np.sum(forecast - actual)
    / np.sum(actual)
) * 100

print("Seasonal Naive Baseline Performance")
print("-----")
print(f"WAPE: {wape_baseline:.2f}%")
print(f"Bias: {bias_baseline:+.2f}%")

```

```

Seasonal Naive Baseline Performance
-----
WAPE: 45.56%
Bias: +13.50%

```

```

# Train Gradient Boosting forecasting model

gb_model = GradientBoostingRegressor(
    n_estimators=200,
    learning_rate=0.05,
    max_depth=3,
    random_state=42
)

gb_model.fit(X_train, y_train)

print("Gradient Boosting model trained successfully!")
print("Training rows:", X_train.shape[0])
print("Number of features:", X_train.shape[1])

```

```

Gradient Boosting model trained successfully!
Training rows: 39000
Number of features: 17

```

```

# Generate predictions for the 6-week test period

y_pred = gb_model.predict(X_test)

print("Predictions generated successfully!")

print("Number of predictions:", len(y_pred))

print("\nFirst 10 predictions:")
print(y_pred[:10])

```

```

Predictions generated successfully!
Number of predictions: 1200

```

First 10 predictions:

```
[19.60138235 18.64173602 19.11140137 19.62543671 18.91248261 18.88394674
19.51396187 19.52793645 18.74662453 19.17257173]
```

```
# Evaluate Gradient Boosting model
```

```
wape_gb = (
    np.sum(np.abs(y_test - y_pred))
    / np.sum(y_test)
) * 100
```

```
bias_gb = (
    np.sum(y_pred - y_test)
    / np.sum(y_test)
) * 100
```

```
print("Gradient Boosting Performance")
print("-----")
print(f"WAPE: {wape_gb:.2f}%")
print(f"Bias: {bias_gb:+.2f}%")
```

```
print("\nComparison with Seasonal Naive")
print("-----")
print(f"Seasonal Naive WAPE: {wape_baseline:.2f}%")
print(f"Gradient Boosting WAPE: {wape_gb:.2f}%")
```

```
Gradient Boosting Performance
```

```
-----
```

```
WAPE: 33.01%
```

```
Bias: +9.55%
```

```
Comparison with Seasonal Naive
```

```
-----
```

```
Seasonal Naive WAPE: 45.56%
```

```
Gradient Boosting WAPE: 33.01%
```

```
# Rolling-origin validation
```

```
validation_weeks = [
    "2025-06-16",
    "2025-07-14",
    "2025-08-11"
]
```

```
validation_results = []
```

```
for origin in validation_weeks:
```

```
    origin = pd.to_datetime(origin)
```

```
    # Training data comes only before the validation period
```

```
    train_cv = model_ready[
        model_ready["week"] < origin
    ].copy()
```



```

# Validate on the next 6 weeks
validation_cv = model_ready[
    (model_ready["week"] >= origin) &
    (model_ready["week"] < origin + pd.Timedelta(weeks=6))
].copy()

X_train_cv = train_cv[feature_columns]
y_train_cv = train_cv["demand"]

X_val_cv = validation_cv[feature_columns]
y_val_cv = validation_cv["demand"]

# Train model
cv_model = GradientBoostingRegressor(
    n_estimators=200,
    learning_rate=0.05,
    max_depth=3,
    random_state=42
)

cv_model.fit(X_train_cv, y_train_cv)

# Predict validation period
y_pred_cv = cv_model.predict(X_val_cv)

# Calculate WAPE
wape_cv = (
    np.sum(np.abs(y_val_cv - y_pred_cv))
    / np.sum(y_val_cv)
) * 100

# Calculate Bias
bias_cv = (
    np.sum(y_pred_cv - y_val_cv)
    / np.sum(y_val_cv)
) * 100

validation_results.append({
    "origin": origin,
    "WAPE": wape_cv,
    "Bias": bias_cv
})

validation_results = pd.DataFrame(validation_results)

print("Rolling-origin validation completed!")

print("\nValidation results:")
print(validation_results)

print("\nAverage WAPE:",
      f"{validation_results['WAPE'].mean():.2f}%")

print("Average Bias:",
      f"{validation_results['Bias'].mean():+.2f}%")

```

Rolling-origin validation completed!

Validation results:

	origin	WAPE	Bias
0	2025-06-16	25.857502	0.757751
1	2025-07-14	26.418844	-0.675923
2	2025-08-11	28.756495	0.694803

Average WAPE: 27.01%

Average Bias: +0.26%

```
# Train final Gradient Boosting model on all available historical data
```

```
X_full = model_ready[feature_columns]
```

```
y_full = model_ready["demand"]
```

```
final_model = GradientBoostingRegressor(  
    n_estimators=200,  
    learning_rate=0.05,  
    max_depth=3,  
    random_state=42  
)
```

```
final_model.fit(X_full, y_full)
```

```
print("Final Gradient Boosting model trained successfully!")
```

```
print("Training rows:", X_full.shape[0])
```

```
print("Number of features:", X_full.shape[1])
```

Final Gradient Boosting model trained successfully!

Training rows: 40200

Number of features: 17

```
# Create the next 6 weeks for forecasting
```

```
latest_week = model_ready["week"].max()
```

```
future_weeks = pd.date_range(  
    start=latest_week + pd.Timedelta(weeks=1),  
    periods=6,  
    freq="7D"  
)
```

```
print("Latest historical week:", latest_week)
```

```
print("\nFuture forecast weeks:")
```

```
for week in future_weeks:  
    print(week)
```

```
print("\nNumber of future weeks:", len(future_weeks))
```

Latest historical week: 2025-10-27 00:00:00

```
Future forecast weeks:
2025-11-03 00:00:00
2025-11-10 00:00:00
2025-11-17 00:00:00
2025-11-24 00:00:00
2025-12-01 00:00:00
2025-12-08 00:00:00
```

```
Number of future weeks: 6
```

```
# Create future SKU-week combinations

future_grid = pd.MultiIndex.from_product(
    [future_weeks, model_ready["sku_id"].unique()],
    names=["week", "sku_id"]
).to_frame(index=False)

print("Future SKU-week grid created!")

print("Shape:", future_grid.shape)

print("Number of weeks:",
      future_grid["week"].nunique())

print("Number of SKUs:",
      future_grid["sku_id"].nunique())

print("\nFirst 10 rows:")
print(future_grid.head(10))
```

```
Future SKU-week grid created!
Shape: (1200, 2)
Number of weeks: 6
Number of SKUs: 200
```

```
First 10 rows:
   week  sku_id
0 2025-11-03  1001
1 2025-11-03  1002
2 2025-11-03  1003
3 2025-11-03  1004
4 2025-11-03  1005
5 2025-11-03  1006
6 2025-11-03  1007
7 2025-11-03  1008
8 2025-11-03  1009
9 2025-11-03  1010
```

```
# Add calendar features to future weeks
```

```
future_grid["week_number"] = (
    future_grid["week"]
    .dt.isocalendar()
    .week
    .astype(int)
)
```

```

future_grid["month"] = future_grid["week"].dt.month
future_grid["quarter"] = future_grid["week"].dt.quarter
future_grid["year"] = future_grid["week"].dt.year

print("Future calendar features created successfully!")

print("\nFirst 10 rows:")
print(
    future_grid[
        [
            "week",
            "sku_id",
            "week_number",
            "month",
            "quarter",
            "year"
        ]
    ].head(10)
)

print("\nMissing values:")
print(
    future_grid[
        [
            "week_number",
            "month",
            "quarter",
            "year"
        ]
    ].isnull().sum()
)

```

Future calendar features created successfully!

First 10 rows:

	week	sku_id	week_number	month	quarter	year
0	2025-11-03	1001	45	11	4	2025
1	2025-11-03	1002	45	11	4	2025
2	2025-11-03	1003	45	11	4	2025
3	2025-11-03	1004	45	11	4	2025
4	2025-11-03	1005	45	11	4	2025
5	2025-11-03	1006	45	11	4	2025
6	2025-11-03	1007	45	11	4	2025
7	2025-11-03	1008	45	11	4	2025
8	2025-11-03	1009	45	11	4	2025
9	2025-11-03	1010	45	11	4	2025

Missing values:

```

week_number    0
month          0
quarter        0
year           0
dtype: int64

```

```
# Add promotion features to future weeks

future_grid["promo_flag"] = 0
future_grid["promo_discount"] = 0
future_grid["promo_count"] = 0

print("Future promotion features created successfully!")

print("\nPromotion feature values:")
print(
    future_grid[
        ["promo_flag", "promo_discount", "promo_count"]
    ].drop_duplicates()
)

print("\nMissing values:")
print(
    future_grid[
        ["promo_flag", "promo_discount", "promo_count"]
    ].isnull().sum()
)
```

Future promotion features created successfully!

Promotion feature values:

	promo_flag	promo_discount	promo_count
	0	0	0

Missing values:

promo_flag	0
promo_discount	0
promo_count	0

dtype: int64

```
# Prepare historical demand for recursive forecasting
```

```
history = weekly_sales_complete[
    ["week", "sku_id", "demand"]
].copy()

history = history.sort_values(
    ["sku_id", "week"]
).reset_index(drop=True)

history_recursive = history.copy()

print("Historical demand prepared for recursive forecasting!")

print("Shape:", history_recursive.shape)

print("\nDate range:")
print("Start:", history_recursive["week"].min())
```

```

print("End:", history_recursive["week"].max())

print("\nNumber of SKUs:",
      history_recursive["sku_id"].nunique())

print("\nLast 5 historical records:")
print(history_recursive.tail())

```

Historical demand prepared for recursive forecasting!
 Shape: (50600, 3)

Date range:
 Start: 2020-12-28 00:00:00
 End: 2025-10-27 00:00:00

Number of SKUs: 200

Last 5 historical records:

	week	sku_id	demand
50595	2025-09-29	1200	7
50596	2025-10-06	1200	20
50597	2025-10-13	1200	15
50598	2025-10-20	1200	19
50599	2025-10-27	1200	26

```

# Recursive 6-week forecasting

forecast_results = []

for future_week in future_weeks:

    future_rows = []

    for sku in history_recursive["sku_id"].unique():

        # Get historical demand for this SKU
        sku_history = history_recursive[
            history_recursive["sku_id"] == sku
        ].sort_values("week")

        demand_values = sku_history["demand"].values

        # Lag features
        lag_1 = demand_values[-1]
        lag_2 = demand_values[-2]
        lag_4 = demand_values[-4]
        lag_8 = demand_values[-8]
        lag_13 = demand_values[-13]
        lag_26 = demand_values[-26]
        lag_52 = demand_values[-52]

        # Rolling mean features
        rolling_mean_4 = np.mean(demand_values[-4:])
        rolling_mean_8 = np.mean(demand_values[-8:])

```

```

rolling_mean_13 = np.mean(demand_values[-13:])

# Calendar features
week_number = future_week.isocalendar().week
month = future_week.month
quarter = future_week.quarter
year = future_week.year

# Promotion features
promo_flag = 0
promo_discount = 0
promo_count = 0

# Create model input
X_future = pd.DataFrame([
    "lag_1": lag_1,
    "lag_2": lag_2,
    "lag_4": lag_4,
    "lag_8": lag_8,
    "lag_13": lag_13,
    "lag_26": lag_26,
    "lag_52": lag_52,
    "rolling_mean_4": rolling_mean_4,
    "rolling_mean_8": rolling_mean_8,
    "rolling_mean_13": rolling_mean_13,
    "week_number": week_number,
    "month": month,
    "quarter": quarter,
    "year": year,
    "promo_flag": promo_flag,
    "promo_discount": promo_discount,
    "promo_count": promo_count
])

# Predict demand
prediction = final_model.predict(X_future)[0]

# Demand cannot be negative
prediction = max(0, prediction)

future_rows.append({
    "week": future_week,
    "sku_id": sku,
    "forecast_demand": prediction
})

# Store predictions for this week
week_forecasts = pd.DataFrame(future_rows)

forecast_results.append(week_forecasts)

# Add predictions to history for the next recursive step
predicted_history = week_forecasts.rename(
    columns={"forecast_demand": "demand"}
)

```

```

        history_recursive = pd.concat(
            [history_recursive, predicted_history],
            ignore_index=True
        )

# Combine all six weeks
future_forecast = pd.concat(
    forecast_results,
    ignore_index=True
)

print("Recursive 6-week forecasting completed!")

print("\nForecast shape:", future_forecast.shape)

print("Number of weeks:",
      future_forecast["week"].nunique())

print("Number of SKUs:",
      future_forecast["sku_id"].nunique())

print("\nFirst 10 forecasts:")
print(future_forecast.head(10))

```

Recursive 6-week forecasting completed!

Forecast shape: (1200, 3)

Number of weeks: 6

Number of SKUs: 200

First 10 forecasts:

	week	sku_id	forecast_demand
0	2025-11-03	1001	18.667519
1	2025-11-03	1002	18.085064
2	2025-11-03	1003	17.815627
3	2025-11-03	1004	18.231688
4	2025-11-03	1005	18.775419
5	2025-11-03	1006	18.018202
6	2025-11-03	1007	18.317055
7	2025-11-03	1008	17.897878
8	2025-11-03	1009	18.015264
9	2025-11-03	1010	17.938547

Forecast sanity check

```

print("Overall forecast statistics:")
print(
    future_forecast["forecast_demand"].describe()
)

print("\nForecast summary by week:")

weekly_forecast_summary = (
    future_forecast
    .groupby("week")["forecast_demand"]

```



```

        .agg(
            total="sum",
            average="mean",
            minimum="min",
            maximum="max"
        )
        .reset_index()
    )

print(weekly_forecast_summary)

print("\nForecast range across all SKU-week predictions:")
print(
    "Minimum:",
    future_forecast["forecast_demand"].min()
)

print(
    "Maximum:",
    future_forecast["forecast_demand"].max()
)

```

Overall forecast statistics:

```

count    1200.000000
mean      19.812327
std       2.127611
min       17.540106
25%       18.323064
50%       18.446601
75%       22.648036
max       27.355193

```

Name: forecast_demand, dtype: float64

Forecast summary by week:

	week	total	average	minimum	maximum
0	2025-11-03	3642.000554	18.210003	17.724738	19.214213
1	2025-11-10	3671.938078	18.359690	18.056441	18.879695
2	2025-11-17	3665.947681	18.329738	17.570169	18.724592
3	2025-11-24	3694.828501	18.474143	17.540106	20.379747
4	2025-12-01	4557.745455	22.788727	20.718754	27.209011
5	2025-12-08	4542.331786	22.711659	18.053201	27.355193

Forecast range across all SKU-week predictions:

Minimum: 17.540105690054208

Maximum: 27.355192821253944

Add SKU ID as a model feature

```
improved_feature_columns = feature_columns + ["sku_id"]
```

```
print("Improved feature set created!")
```

```
print("Number of features:", len(improved_feature_columns))
```

```
print("\nNew feature added:")
```

```
print("- sku_id")
```

Improved feature set created!
Number of features: 18

New feature added:
- sku_id

```
# Prepare training data with SKU ID

X_full_improved = model_ready[improved_feature_columns]
y_full_improved = model_ready["demand"]

# Train improved Gradient Boosting model

improved_model = GradientBoostingRegressor(
    n_estimators=200,
    learning_rate=0.05,
    max_depth=3,
    random_state=42
)

improved_model.fit(
    X_full_improved,
    y_full_improved
)

print("Improved Gradient Boosting model trained successfully!")

print("Training rows:", X_full_improved.shape[0])
print("Number of features:", X_full_improved.shape[1])
```

Improved Gradient Boosting model trained successfully!
Training rows: 40200
Number of features: 18

```
# Evaluate the improved model on the 6-week test period

X_test_improved = test_data[improved_feature_columns]
y_test_improved = test_data["demand"]

# Generate predictions
y_pred_improved = improved_model.predict(X_test_improved)

# Calculate WAPE
wape_improved = (
    np.sum(np.abs(y_test_improved - y_pred_improved))
    / np.sum(y_test_improved)
) * 100

# Calculate Bias
bias_improved = (
    np.sum(y_pred_improved - y_test_improved)
    / np.sum(y_test_improved)
) * 100
```

```

print("Improved Gradient Boosting Performance")
print("-----")
print(f"WAPE: {wape_improved:.2f}%")
print(f"Bias: {bias_improved:+.2f}%")

print("\nModel comparison")
print("-----")
print(f"Seasonal Naive WAPE: {wape_baseline:.2f}%")
print(f"Original Gradient Boosting WAPE: {wape_gb:.2f}%")
print(f"Improved Gradient Boosting WAPE: {wape_improved:.2f}%")

```

Improved Gradient Boosting Performance

WAPE: 31.73%

Bias: +4.57%

Model comparison

Seasonal Naive WAPE: 45.56%

Original Gradient Boosting WAPE: 33.01%

Improved Gradient Boosting WAPE: 31.73%

```
# Check improved model prediction distribution
```

```

print("Improved model prediction statistics:")
print(
    pd.Series(y_pred_improved).describe()
)

```

```

print("\nPrediction range:")
print("Minimum:", y_pred_improved.min())
print("Maximum:", y_pred_improved.max())

```

```

print("\nFirst 10 improved predictions:")
print(y_pred_improved[:10])

```

Improved model prediction statistics:

```

count    1200.000000
mean      18.634615
std       0.311960
min       17.826443
25%       18.408937
50%       18.602040
75%       18.836021
max       19.988055
dtype: float64

```

Prediction range:

Minimum: 17.826443494169947

Maximum: 19.988055254289694

First 10 improved predictions:

```

[18.94532566 18.11785127 18.43547868 18.80102929 18.53679767 18.30474032
 19.04313673 18.58492091 18.12351105 18.29514728]

```

```

print("Actual demand statistics:")
print(y_test_improved.describe())

print("\nImproved model prediction statistics:")
print(pd.Series(y_pred_improved).describe())

print("\nComparison")
print("-----")
print(f"Actual mean:      {y_test_improved.mean():.2f}")
print(f"Predicted mean:   {y_pred_improved.mean():.2f}")
print(f"Actual std:       {y_test_improved.std():.2f}")
print(f"Predicted std:    {y_pred_improved.std():.2f}")
print(f"Actual minimum:   {y_test_improved.min():.2f}")
print(f"Predicted minimum: {y_pred_improved.min():.2f}")
print(f"Actual maximum:   {y_test_improved.max():.2f}")
print(f"Predicted maximum: {y_pred_improved.max():.2f}")

```

```

model_data = weekly_sales_complete.copy()

model_data = (
    model_data
    .sort_values(["sku_id", "week"])
    .reset_index(drop=True)
)

print("model_data restored successfully!")
print("Shape:", model_data.shape)
print("Columns:", len(model_data.columns))

```

```

model_data restored successfully!
Shape: (50600, 28)
Columns: 28

```

```

model_data["sku_mean_demand"] = (model_data.groupby("sku_id")["demand"].transform(
    lambda x: x.rolling(7).mean()
))

print("SKU historical mean feature created.")
print(model_data[["week", "sku_id", "demand", "sku_mean_demand"]].head(10))

```

```

SKU historical mean feature created.

```

	week	sku_id	demand	sku_mean_demand
0	2020-12-28	1001	39	NaN
1	2021-01-04	1001	40	39.000000
2	2021-01-11	1001	52	39.500000
3	2021-01-18	1001	45	43.666667
4	2021-01-25	1001	64	44.000000
5	2021-02-01	1001	35	48.000000
6	2021-02-08	1001	88	45.833333
7	2021-02-15	1001	47	51.857143

8	2021-02-22	1001	61	51.250000
9	2021-03-01	1001	19	52.333333

```
improved_feature_columns_v2 = improved_feature_columns + ["sku_mean_demand"]

print("Number of features:", len(improved_feature_columns_v2))
print("\nFeatures:")
print(improved_feature_columns_v2)
```

Number of features: 19

Features:
['lag_1', 'lag_2', 'lag_4', 'lag_8', 'lag_13', 'lag_26', 'lag_52', 'rolling_m

```
# Add the new feature to the feature list
feature_columns_v2 = improved_feature_columns_v2

# Remove rows where required features are unavailable
model_ready_v2 = model_data.dropna(
    subset=feature_columns_v2
).copy()

# Use the same final 6 weeks as before
test_weeks_v2 = sorted(model_ready_v2["week"].unique())[-6:]

# Time-based train/test split
train_data_v2 = model_ready_v2[
    ~model_ready_v2["week"].isin(test_weeks_v2)
].copy()

test_data_v2 = model_ready_v2[
    model_ready_v2["week"].isin(test_weeks_v2)
].copy()

print("Model-ready shape:", model_ready_v2.shape)
print("Training shape:", train_data_v2.shape)
print("Test shape:", test_data_v2.shape)

print("\nTest weeks:")
print(test_weeks_v2)
```

Model-ready shape: (40200, 29)
Training shape: (39000, 29)
Test shape: (1200, 29)

Test weeks:
[Timestamp('2025-09-22 00:00:00'), Timestamp('2025-09-29 00:00:00'), Timestamp

```
X_train_v2 = train_data_v2[feature_columns_v2]
y_train_v2 = train_data_v2["demand"]
```

```

X_test_v2 = test_data_v2[feature_columns_v2]
y_test_v2 = test_data_v2["demand"]

model_v2 = GradientBoostingRegressor(
    n_estimators=200,
    learning_rate=0.05,
    max_depth=3,
    random_state=42
)

model_v2.fit(X_train_v2, y_train_v2)

print("Model V2 trained successfully!")
print("Training features:", X_train_v2.shape)
print("Test features:", X_test_v2.shape)

```

```

Model V2 trained successfully!
Training features: (39000, 19)
Test features: (1200, 19)

```

Double-click (or enter) to edit

```

# Generate predictions
y_pred_v2 = model_v2.predict(X_test_v2)

# Calculate WAPE
wape_v2 = (
    np.sum(np.abs(y_test_v2 - y_pred_v2))
    / np.sum(y_test_v2)
) * 100

# Calculate Bias
bias_v2 = (
    np.sum(y_pred_v2 - y_test_v2)
    / np.sum(y_test_v2)
) * 100

print("Model V2 Performance")
print("-----")
print(f"WAPE: {wape_v2:.2f}%")
print(f"Bias: {bias_v2:+.2f}%")

print("\nModel comparison")
print("-----")
print(f"Seasonal Naive WAPE: {wape_baseline:.2f}%")
print(f"Original Gradient Boosting WAPE: {wape_gb:.2f}%")
print(f"Improved Gradient Boosting WAPE: {wape_improved:.2f}%")
print(f"Model V2 WAPE: {wape_v2:.2f}%")

```

```

Model V2 Performance
-----
WAPE: 32.85%
Bias: +9.05%

```

Model comparison

```
-----  
Seasonal Naive WAPE:          45.56%  
Original Gradient Boosting WAPE: 33.01%  
Improved Gradient Boosting WAPE: 31.73%  
Model V2 WAPE:              32.85%
```

```
feature_importance = pd.DataFrame({  
    "feature": improved_feature_columns,  
    "importance": improved_model.feature_importances_  
})  
  
feature_importance = feature_importance.sort_values(  
    "importance",  
    ascending=False  
).reset_index(drop=True)  
  
print("Feature Importance")  
print("-----")  
print(feature_importance)
```

Feature Importance

```
-----  
      feature  importance  
0      promo_flag    0.269600  
1      promo_count    0.228790  
2      promo_discount    0.225293  
3      week_number    0.200586  
4          month    0.035014  
5          year    0.010535  
6          lag_52    0.008330  
7          quarter    0.007093  
8          lag_4    0.002464  
9          lag_26    0.002463  
10         lag_1    0.002397  
11  rolling_mean_8    0.001915  
12  rolling_mean_13    0.001345  
13  rolling_mean_4    0.001234  
14         lag_13    0.001096  
15         lag_2    0.001068  
16         lag_8    0.000537  
17         sku_id    0.000240
```

```
print("Lag 52 statistics")  
print("-----")  
print(model_ready["lag_52"].describe())  
  
print("\nLag 52 vs Actual Demand")  
print("-----")  
  
lag52_corr = model_ready[["lag_52", "demand"]].corr().iloc[0, 1]  
  
print(f"Correlation: {lag52_corr:.4f}")
```

```

print("\nSKU-level lag_52 variation")
print("-----")

sku_lag52_stats = (
    model_ready
    .groupby("sku_id")["lag_52"]
    .agg(["mean", "std", "min", "max"])
)

print(sku_lag52_stats.head(10))

```

Lag 52 statistics

```

-----
count      40200.000000
mean         33.259975
std         17.009675
min          1.000000
25%         20.000000
50%         30.000000
75%         44.000000
max        131.000000
Name: lag_52, dtype: float64

```

Lag 52 vs Actual Demand

```

-----
Correlation: 0.4891

```

SKU-level lag_52 variation

```

-----

```

	mean	std	min	max
sku_id				
1001	33.696517	17.965034	6.0	95.0
1002	33.980100	17.101158	6.0	82.0
1003	34.019900	16.769007	7.0	96.0
1004	32.631841	16.784034	5.0	99.0
1005	33.592040	17.542598	6.0	100.0
1006	32.303483	15.830428	9.0	75.0
1007	33.139303	17.276588	4.0	96.0
1008	33.139303	17.057857	6.0	92.0
1009	33.601990	17.714141	8.0	97.0
1010	33.656716	18.158099	3.0	89.0

```

# Test a simple blended forecasting approach

```

```

blend_predictions = (
    0.7 * test_data["lag_52"].values
    + 0.3 * test_data["rolling_mean_4"].values
)

```

```

wape_blend = (
    np.sum(np.abs(y_test.values - blend_predictions))
    / np.sum(y_test.values)
) * 100

```

```

bias_blend = (
    np.sum(blend_predictions - y_test.values)
    / np.sum(y_test.values)
)

```



```
) * 100
```

```
print("Lag-52 + Recent-Demand Blend")
print("-----")
print(f"WAPE: {wape_blend:.2f}%")
print(f"Bias: {bias_blend:+.2f}%")

print("\nComparison")
print("-----")
print(f"Seasonal Naive WAPE:           {wape_baseline:.2f}%")
print(f"Original Gradient Boosting:     {wape_gb:.2f}%")
print(f"Improved Gradient Boosting:      {wape_improved:.2f}%")
print(f"Lag-52 + Recent Blend:         {wape_blend:.2f}%")
```

Lag-52 + Recent-Demand Blend

WAPE: 40.15%

Bias: +12.32%

Comparison

Seasonal Naive WAPE: 45.56%

Original Gradient Boosting: 33.01%

Improved Gradient Boosting: 31.73%

Lag-52 + Recent Blend: 40.15%

```
final_weeks_check = (
    weekly_sales_complete
    .groupby("week", as_index=False)["demand"]
    .sum()
    .sort_values("week")
    .tail(8)
)
```

```
print("Last 8 weekly demand totals:")
```

```
print(final_weeks_check)
```

```
print("\nFinal historical week:")
```

```
print(weekly_sales_complete["week"].max())
```

Last 8 weekly demand totals:

	week	demand
245	2025-09-08	3882
246	2025-09-15	3464
247	2025-09-22	3690
248	2025-09-29	3796
249	2025-10-06	4086
250	2025-10-13	3020
251	2025-10-20	3858
252	2025-10-27	2934

Final historical week:

2025-10-27 00:00:00

```
# Final winning model
```

```
final_model = GradientBoostingRegressor(
```

```

        n_estimators=200,
        learning_rate=0.05,
        max_depth=3,
        random_state=42
    )

X_full_final = model_ready[improved_feature_columns]
y_full_final = model_ready["demand"]

final_model.fit(X_full_final, y_full_final)

print("Final winning model trained successfully!")
print("Training data:", X_full_final.shape)
print("Features:", len(improved_feature_columns))

```

```

Final winning model trained successfully!
Training data: (40200, 18)
Features: 18

```

```

# Get the latest historical week
latest_week = weekly_sales_complete["week"].max()

# Generate next 6 weeks
future_weeks = pd.date_range(
    start=latest_week + pd.Timedelta(weeks=1),
    periods=6,
    freq="7D"
)

print("Latest historical week:", latest_week)

print("\nFuture forecast weeks:")
for week in future_weeks:
    print(week.date())

```

```

Latest historical week: 2025-10-27 00:00:00

Future forecast weeks:
2025-11-03
2025-11-10
2025-11-17
2025-11-24
2025-12-01
2025-12-08

```

```

# Create all SKU × future week combinations

all_skus = weekly_sales_complete["sku_id"].unique()

future_grid = pd.MultiIndex.from_product(
    [future_weeks, all_skus],
    names=["week", "sku_id"]
).to_frame(index=False)

print("Future grid created successfully!")

```

```
print("Shape:", future_grid.shape)
print("Number of weeks:", future_grid["week"].nunique())
print("Number of SKUs:", future_grid["sku_id"].nunique())

print("\nFirst 10 rows:")
print(future_grid.head(10))
```

```
Future grid created successfully!
Shape: (1200, 2)
Number of weeks: 6
Number of SKUs: 200
```

```
First 10 rows:
   week  sku_id
0 2025-11-03   1001
1 2025-11-03   1002
2 2025-11-03   1003
3 2025-11-03   1004
4 2025-11-03   1005
5 2025-11-03   1006
6 2025-11-03   1007
7 2025-11-03   1008
8 2025-11-03   1009
9 2025-11-03   1010
```

```
# Add calendar features to future grid

future_data = future_grid.copy()

future_data["week_number"] = (
    future_data["week"].dt.isocalendar().week.astype(int)
)

future_data["month"] = future_data["week"].dt.month
future_data["quarter"] = future_data["week"].dt.quarter
future_data["year"] = future_data["week"].dt.year

print("Future calendar features added successfully!")

print("\nSample:")
print(
    future_data[
        ["week", "sku_id", "week_number", "month", "quarter", "year"]
    ].head(10)
)
```

```
Future calendar features added successfully!
```

```
Sample:
   week  sku_id  week_number  month  quarter  year
0 2025-11-03   1001         45    11         4  2025
1 2025-11-03   1002         45    11         4  2025
2 2025-11-03   1003         45    11         4  2025
3 2025-11-03   1004         45    11         4  2025
4 2025-11-03   1005         45    11         4  2025
5 2025-11-03   1006         45    11         4  2025
6 2025-11-03   1007         45    11         4  2025
```

7	2025-11-03	1008	45	11	4	2025
8	2025-11-03	1009	45	11	4	2025
9	2025-11-03	1010	45	11	4	2025

```
# Add promotion information to future data
```

```
future_data = future_data.merge(
    promo_weekly_agg,
    on="week",
    how="left"
)
```

```
# No promotion = 0
```

```
future_data["promo_flag"] = future_data["promo_flag"].fillna(0)
future_data["promo_discount"] = future_data["promo_discount"].fillna(0)
future_data["promo_count"] = future_data["promo_count"].fillna(0)
```

```
print("Future promotion features added successfully!")
```

```
print("\nPromotion feature summary:")
```

```
print(
    future_data[
        ["promo_flag", "promo_discount", "promo_count"]
    ].drop_duplicates()
)
```

```
print("\nMissing values:")
```

```
print(
    future_data[
        ["promo_flag", "promo_discount", "promo_count"]
    ].isna().sum()
)
```

```
Future promotion features added successfully!
```

```
Promotion feature summary:
```

```
promo_flag  promo_discount  promo_count
0           0.0           0.0           0.0
```

```
Missing values:
```

```
promo_flag      0
promo_discount  0
promo_count     0
dtype: int64
```

```
# Prepare historical demand for recursive forecasting
```

```
history = weekly_sales_complete[
    ["week", "sku_id", "demand"]
].copy()
```

```
history = (
    history
    .sort_values(["sku_id", "week"])
)
```

```

        .reset_index(drop=True)
    )

    # Copy so that future predictions can be appended
    history_recursive = history.copy()

    print("Historical demand prepared successfully!")

    print("Shape:", history_recursive.shape)
    print("Number of SKUs:", history_recursive["sku_id"].nunique())
    print("Historical start:", history_recursive["week"].min())
    print("Historical end:", history_recursive["week"].max())

```

```

Historical demand prepared successfully!
Shape: (50600, 3)
Number of SKUs: 200
Historical start: 2020-12-28 00:00:00
Historical end: 2025-10-27 00:00:00

```

```

# Generate recursive 6-week forecast using the winning model

forecast_results = []

for future_week in future_weeks:

    week_predictions = []

    for sku in all_skus:

        # Get historical + previously predicted demand for this SKU
        sku_history = history_recursive[
            history_recursive["sku_id"] == sku
        ].sort_values("week")

        demand_history = sku_history["demand"].values

        # Create lag features
        lag_1 = demand_history[-1]
        lag_2 = demand_history[-2]
        lag_4 = demand_history[-4]
        lag_8 = demand_history[-8]
        lag_13 = demand_history[-13]
        lag_26 = demand_history[-26]
        lag_52 = demand_history[-52]

        # Create rolling features using previous demand only
        rolling_mean_4 = np.mean(demand_history[-4:])
        rolling_mean_8 = np.mean(demand_history[-8:])
        rolling_mean_13 = np.mean(demand_history[-13:])

        # Get calendar features
        week_number = future_week.isocalendar().week
        month = future_week.month
        quarter = future_week.quarter
        year = future_week.year

```

```

# Future promotion features
promo_row = future_data[
    (future_data["week"] == future_week) &
    (future_data["sku_id"] == sku)
].iloc[0]

promo_flag = promo_row["promo_flag"]
promo_discount = promo_row["promo_discount"]
promo_count = promo_row["promo_count"]

# Create feature row in exactly the same order as training
X_future = pd.DataFrame([
    "lag_1": lag_1,
    "lag_2": lag_2,
    "lag_4": lag_4,
    "lag_8": lag_8,
    "lag_13": lag_13,
    "lag_26": lag_26,
    "lag_52": lag_52,
    "rolling_mean_4": rolling_mean_4,
    "rolling_mean_8": rolling_mean_8,
    "rolling_mean_13": rolling_mean_13,
    "week_number": week_number,
    "month": month,
    "quarter": quarter,
    "year": year,
    "promo_flag": promo_flag,
    "promo_discount": promo_discount,
    "promo_count": promo_count,
    "sku_id": sku
])

# Predict
prediction = final_model.predict(
    X_future[improved_feature_columns]
)[0]

# Prevent negative demand
prediction = max(0, prediction)

week_predictions.append({
    "week": future_week,
    "sku_id": sku,
    "forecast_demand": prediction
})

# Add prediction to history for next recursive step
history_recursive = pd.concat([
    history_recursive,
    pd.DataFrame([
        "week": future_week,
        "sku_id": sku,
        "demand": prediction
    ])
])

```

```

    ], ignore_index=True)

    forecast_results.extend(week_predictions)

future_forecast = pd.DataFrame(forecast_results)

print("6-week recursive forecast generated successfully!")
print("Shape:", future_forecast.shape)

print("\nForecast weeks:", future_forecast["week"].nunique())
print("SKUs:", future_forecast["sku_id"].nunique())

print("\nFirst 10 predictions:")
print(future_forecast.head(10))

```

6-week recursive forecast generated successfully!
Shape: (1200, 3)

Forecast weeks: 6
SKUs: 200

First 10 predictions:

	week	sku_id	forecast_demand
0	2025-11-03	1001	18.688963
1	2025-11-03	1002	18.268207
2	2025-11-03	1003	18.113649
3	2025-11-03	1004	18.417987
4	2025-11-03	1005	18.778659
5	2025-11-03	1006	18.201344
6	2025-11-03	1007	18.445166
7	2025-11-03	1008	18.084599
8	2025-11-03	1009	18.203883
9	2025-11-03	1010	18.147021

```

print("Overall Forecast Statistics")
print("-----")
print(future_forecast["forecast_demand"].describe())

print("\nWeekly Forecast Summary")
print("-----")

weekly_forecast_summary = (
    future_forecast
    .groupby("week")["forecast_demand"]
    .agg(["sum", "mean", "min", "max"])
    .reset_index()
)

print(weekly_forecast_summary)

print("\nForecast missing values:")
print(future_forecast.isna().sum())

```

Overall Forecast Statistics

count 1200.000000

```
mean      19.968946
std       2.098964
min       17.700827
25%      18.494872
50%      18.601209
75%      22.815589
max       27.223239
Name: forecast_demand, dtype: float64
```

Weekly Forecast Summary

```
-----
           week      sum      mean      min      max
0 2025-11-03  3672.082871  18.360414  17.946594  19.257528
1 2025-11-10  3707.218007  18.536090  18.284249  19.048549
2 2025-11-17  3698.988162  18.494941  17.714191  18.918437
3 2025-11-24  3729.322420  18.646612  17.700827  20.444522
4 2025-12-01  4586.520966  22.932605  21.306525  27.223239
5 2025-12-08  4568.602501  22.843013  21.372308  24.715701
```

Forecast missing values:

```
week      0
sku_id    0
forecast_demand  0
dtype: int64
```

```
# Save final 6-week forecast
```

```
future_forecast.to_csv(
    "final_6_week_forecast.csv",
    index=False
)

print("Final forecast saved successfully!")
print("File: final_6_week_forecast.csv")
print("Rows:", len(future_forecast))
```

```
Final forecast saved successfully!
File: final_6_week_forecast.csv
Rows: 1200
```

```
from google.colab import files
```

```
files.download("final_6_week_forecast.csv")
```